



Comunicación por paso de mensajes en SD

Contenido

Objetivo de la Práctica	1
Introducción	1
Preguntas Teóricas	2
Ejercicio 1 – Comunicación síncrona con REST	2
Ejercicio 2 – Comunicación asíncrona con RabbitMQ	2
Ejercicio 3 – Procesamiento de eventos con Kafka	2

Objetivo de la Práctica

El objetivo de esta práctica es que los estudiantes **apliquen los conceptos teóricos de comunicación en sistemas distribuidos mediante paso de mensajes a casos prácticos actuales**, utilizando tecnologías vigentes en la industria.

A través de la resolución de ejercicios, los estudiantes:

- **Comprenderán** las diferencias entre comunicación síncrona y asíncrona en entornos distribuidos.
- **Experimentarán** con la implementación de servicios que se comunican entre sí mediante diferentes enfoques de mensajería.
- **Utilizarán tecnologías actuales** como **gRPC, RabbitMQ y Apache Kafka**, observando sus ventajas, limitaciones y casos de uso.
- **Relacionarán la teoría con la práctica**, identificando problemas comunes (latencia, pérdida de mensajes, tolerancia a fallos, desacoplamiento entre servicios) y analizando cómo cada tecnología los aborda.
- **Desarrollarán criterio técnico** para seleccionar la herramienta adecuada según el tipo de comunicación requerida (síncrona vs. asíncrona, request/response vs. event streaming).

Introducción

En los sistemas distribuidos, los procesos que se ejecutan en diferentes nodos necesitan intercambiar información para coordinarse, compartir datos o realizar tareas colaborativas. Una de las formas más utilizadas para esta comunicación es el **paso de mensajes**.



Este enfoque evita el uso de memoria compartida y se basa en el envío de mensajes a través de redes de comunicación, con tecnologías como **REST**, **gRPC**, **WebSockets**, **RabbitMQ**, **Kafka**, **ZeroMQ**, entre otras.

Preguntas Teóricas

1. ¿Qué ventajas tiene el paso de mensajes frente a la memoria compartida en sistemas distribuidos?
2. Explica las diferencias entre **comunicación síncrona** y **asíncrona** por mensajes.
3. ¿Qué problemas pueden aparecer en la comunicación por mensajes? (ej: pérdida, duplicación, orden, latencia).
4. ¿Cuál es la diferencia entre un **Message Queue** (ej. RabbitMQ) y un **Event Streaming Platform** (ej. Kafka) ?
5. ¿Qué rol cumple un **middleware de mensajería** en este tipo de sistemas?

Ejercicio 1 – Comunicación síncrona con REST

Implementar un prototipo donde OrdersService consulta a InventoryService si un producto está disponible antes de confirmar el pedido.

- Tecnologías: **REST**
- Compare gRPC frente a REST en este escenario.

Ejercicio 2 – Comunicación asíncrona con RabbitMQ

Implementar una cola de mensajes donde OrdersService publique un mensaje de "Pedido Creado" y NotificationService lo consuma para enviar un correo al cliente.

- Tecnologías: **RabbitMQ / Docker**
- Pregunta: ¿qué ocurre si NotificationService está caído temporalmente?

Ejercicio 3 – Procesamiento de eventos con Kafka

Extender el sistema para que cada cambio de estado del pedido (creado, confirmado, enviado) se publique en un **topic de Kafka** y pueda ser consumido tanto por NotificationService como por un nuevo AnalyticsService.

- Tecnologías: **Kafka + Docker Compose**



- Pregunta: ¿qué diferencias notarías entre usar Kafka y RabbitMQ en este escenario?